

Assignment 1: Lexing

due: 2/6

In this assignment we will use WrangLR to create a lexer for MiniJava. We haven't really gotten into the details of MiniJava yet, but don't worry. All we really need to know are the tokens. You'll also want to have an ASCII table on hand. You can find one here <https://www.asciitable.com/>. You can download the asst1.zip file that has the starter code you will need.

Included in asst1.zip you'll find

- **src** The source code files
 - **main** Include Main.java to run the program
 - **parse** the files for making the lexer.
 - * **TokenGrammar.java** the lexer
 - **errorMsg** files for tracking error messages.
 - * **CompWarning.java**: General Warning object.
Inside you'll find a single method to construct a `CommentInComment()` warning.
(ex: `/* is /* a comment */ in a comment */`)
 - * **Error.java**: Generic Error object.
Inside this class you'll find several methods to create error messages.
 - `bigChar()`
A character literal with more than one character. (ex: `'abc'`)
 - `outOfRange()`
An integer literal that's too big. (ex: `1234567890987654321`)
 - `emptyChar()`
a character literal that doesn't contain a character (ex: `''`)
 - `unterminatedChar()`
A character literal that isn't properly terminated (ex: `'a`)
 - `unterminatedMultiComment()`
A `/*` comment that goes to the end of the file.
 - `unterminatedSingleLineComment()`
A `//` comment on the last line of the file.
 - `illegalChar()`
An illegal character.

- `unterminatedString()`
A string that does not have a closing quote (ex: `"foo`).

- `Test0.java` and `Test1.java`

You only need to modify `TokenGrammar.java` for this assignment, although you're encouraged to look around at the other files. You will need the different `CompError` and `CompWarning` method in `errorMsg`. The goal is to give a definition for all of MiniJava's tokens. The first part of the file is filled with helper methods and the definition for `token`.

You don't need to modify anything until you see the comment

```
////////////////////////////////////
// Your modifications should start here
////////////////////////////////////
```

At the end of the file you will find a bunch of dummy definitions for tokens. You need to fill in a real definition for each one. I recommend deleting the dummy definition after you've filled in the real one. This will make fewer states for `WrangLR`, and will make your compile times a lot shorter.

MiniJava tokens

MiniJava has a bunch of tokens we need to recognize. They come in 5 groups: identifiers, numbers, keywords, identifiers, and operators. The keywords and operators are the easiest group.

Keywords:

```
boolean, break, class, else, extends,
false, for, if, instanceof, int,
new, null, public, return, super,
switch, this, true, void, while
```

Operators:

```
&& ||
!= == < <= > >=
+ - * / %
! -- ++ . , : ;
( ) [ ] { } =
```

Along with the keywords, there are a bunch of Java keywords that we would like to recognize, so we can avoid them in variable names. It'd be weird if someone

named their class “try”.

Disallowed keywords:

```
abstract, assert, byte, case, catch, char, const,
continue, default, do, double, enum, final, finally,
float, goto, implements, import, interface, long,
native, package, private, protected, short, static,
strictfp, synchronized, throw, throws, transient,
try, volatile
```

The convention we’ll use for token definitions is to use a backtick before the name (the backtick key with tilde near esc). So the token for `class` might look like:

```
//: ‘class ::= "class" !idChar white*
```

Notice that we want to make sure there’s no possible identifier character after “class”, or we should read it as an ID. We also use the longest match rule. We should read as many characters as we can into a token. For example `==` should be parsed as one token, not two `=` tokens.

A MiniJava identifier

- starts with a letter
- may contains any number of letters, digits, or underscores.
- must not match any keywords.
- follow the longest match rule.

A MiniJava integer literal

- may be the number “0”
 - may be one or more digits that does not start with a “0”
 - must not be too large to fit in an int
- This is already done in the starter code.

We’ll handle negative numbers when we write the parser.

A MiniJava String Literal

- begins with a double quote character ”
- may contains any number of printable characters not include ” or backslash \
- ends with a ”

A MiniJava character Literal

- begins with a single quote character `'`
- contains exactly one character that's not a `'` or backslash `\`
- ends with a `'`

Strings can include any printable character (characters between 32 and 127 in the ASCII table) This includes spaces letter digits and symbols, but not special characters like tabs or newlines.

Whitespace

The last group of tokens is the most complicated, and it's also the group you should start with. One of the primary purposes of the lexer is to get rid of whitespace so we don't have to worry about it in the parser.

Handling whitespace is important. Take a look at the following code.

```
if(x < 5) { System.out.println(x); }
```

We could write that in any number of different ways.

```
if ( x < 5 ) { System . out . println ( x ) ; }
if(x<5){System.out.println(x);}
if( x < 5 ) {
    System.out.println(x);
}
```

Or, if we're civilized,

```
if(x < 5)
{
    System.out.println(x);
}
```

Really, the rule we need is that every token can be followed by any amount of whitespace. To handle whitespace we will make a rule `white` that will match a whitespace character. We can then alter all of our rules to read any amount of whitespace. For example, the rule:

```
//: '&&' ::= "&&"
```

can become

```
//: '&&' ::= "&&" white*
```

So, and `&&` token is followed by any (possibly empty) amount of whitespace. In fact, this is really what it means to be a token. A string of characters that can't be divided by whitespace.

Whitespace characters you should look for

- space (‘ ’ ASCII 32)
- tab (‘\t’ ASCII 9)
- formfeed (ASCII 12)
- newline (‘\n’ ASCII 10)
- carriage return (‘\r’ ASCII 13)
- Single line comments (comments that start with “//”)
- Multi line comments (“/*” “*/” comments)

Project setup

The starter code will compile on a linux system. I've included 3 scripts to help with compiling and running the program.

- **gen** in the parse directory: runs WrangLR to generate `TokenGrammerParseTable.java`
• **comp**: compiles `main/Main.java` **run**: runs the program on a java file

As an example, assuming you're in the `src` directory, you can run the program with

```
cd parse
./gen
cd ..
./comp
./run ../Test0.java
```

These scripts should work fine for Linux and Mac. I have made `.bat` scripts for windows, and I'm even pretty sure they work.

extensions

If you complete the basic assignment and it works correctly, then you'll end up with a 'B' (85%) on this assignment. To get full points you need to complete some extensions.

- Extension 1: adds 3%
Print a warning message when you see a `/*` inside a `/*` comment.
- Extension 2: adds 6%
Recognize escape sequences in string and character literals
 - `\\` is a single backslash
 - `\"` is a double quote
 - `\'` is a single quote
 - `\n` is a new line (ASCII 10)
 - `\t` is a tab (ASCII 9)
 - `\f` is a form feed (ASCII 12)
 - `\r` is a carriage return (ASCII 13)
- Extension 3: adds 6%
extend `INT_LITERAL` to handle octal and hexadecimal numbers. An octal number must start with a '0' and be comprised of only digits from 0 to 7. A hex number starts with '0x' or '0X' and has digits 0 to 9, and A to F.
- Extension 4: adds 6% (possible partial credit)
Print errors for Illegal characters and ill-formed literals.

- Treat illegal characters as whitespace, but print an error
- If an illegal character is in a string or character literal then print an error
- If a character literal contains 0 or 2+ characters, print an error
- If a string or character is unterminated then print an error. MiniJava does not allow multi-line strings. So if we get to the end of a line, the string is unterminated.
- If a comment is unterminated, then print an error. A comment is unterminated if we reach the end of file while lexing.

I've set up a server for you at `cs358.campus.up.edu` that you can access with `ssh`.

If you're on mac or linux, you probably already have `ssh` installed. If not, you can install it using `apt` or `yum` or `brew` if your on mac. I think windows has an `ssh` built in now, but if not I think `putty` is still a thing.

Anyway, if you're on campus, you can log onto the server with

```
ssh cs358.campus.up.edu
```

Once there, you'll find that your home directory is pretty empty. You should make a folder called `asst1`. Then you can logout and upload your assignment with `scp`. The command is:

```
scp TokenGrammar.java cs358.campus.up.edu:asst1/TokenGrammar.java
```

Once you've uploaded the file, I have a script that runs once an hour. It will run your file through all of the grading scripts, and will put the `Summary.txt` and `Output.txt` files in your `asst1` directory. If you get a `compErr.txt`, then my grading program ran into an error compiling your program.

Note: This is not a development machine. It is a *really* bad idea to try to do this assignment by uploading the file, and waiting an hour to see the grade. You should develop and test yourself, and use this to check to see how you're doing.

Warning: This is a resource I'm providing for you. You do not have permission to poke around in here or try to hack the machine. Any attempt to do so will result in immediate loss of access, and possibly an academic integrity violation.